

4. Система планирования «толстый» клиент-сервер

Введение

Работа 4. «Толстый» клиент с файловой БД

Цель работы — построение распределенной двухуровневой системы управления.

Распределенные (сетевые) системы в последнее время получают все более широкое распространение. В данной работе рассматривается система «сервер БД — сервер приложений — клиенты». В качестве СУБД используется SQLite.

Описание работы

Необходимо по запросу клиента передать ему содержимое таблицы БД из сервера.

Обобщенная программная структура представлена на рисунке 4.1, а детальная — на рисунке 4.2.

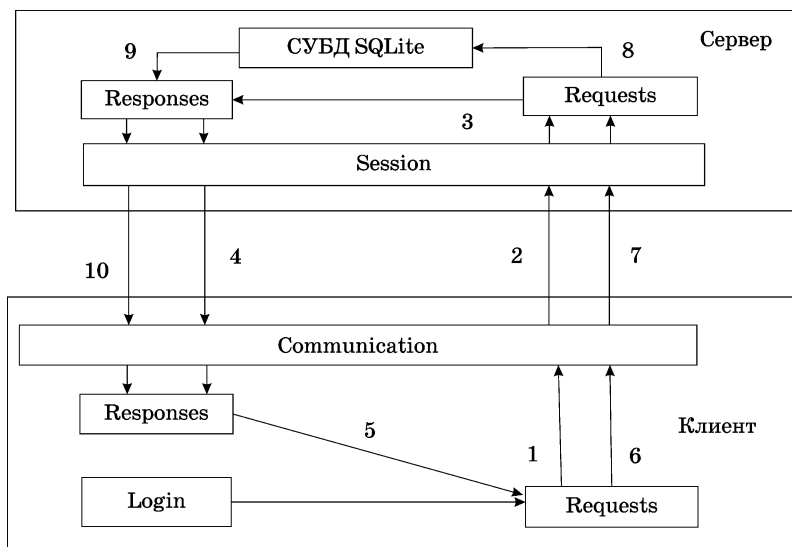


Рис. 4.1

Обобщенная схема системы:
1–4 — запрос и ответ на установление связи; 5–10 — запрос и ответ на выдачу данных.

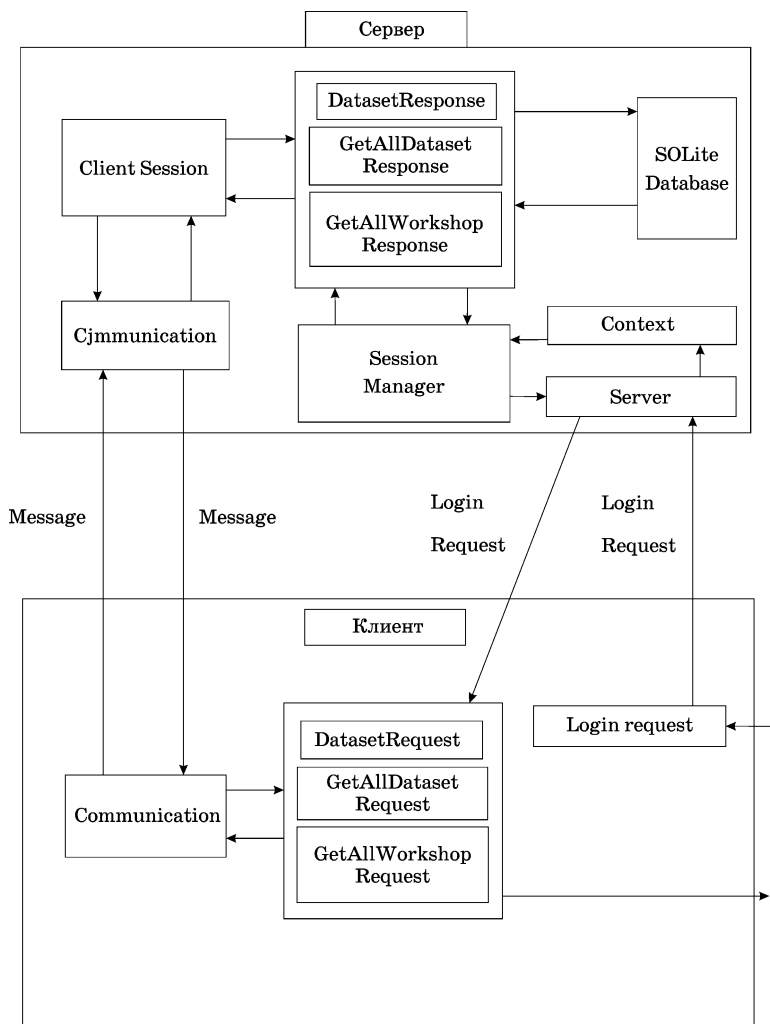


Рис. 4.2

Детальная программная структура

Файлы имеют следующее назначение.

Клиентская часть

Tcp.Core — основной пакет, содержащий клиентскую и серверную части.

Request — описывает необходимые данные для запроса, наследует интерфейс IMessage.

Response — описывает необходимые данные для ответа, наследует интерфейс IMessage.

Следующие три класса будут наследовать Serializable, что поможет передавать данные по программе в виде байт кода.

Dataset Object — содержит в себе объекты данных, т. е. матрицы, имена.

Login Object — объекты, необходимые для входа в систему, т. е. логин и пароль, роль.

Workshop Object — содержит данные, изменяемые в ходе работы.

Пакет Request выполняет следующие функции:

- Dataset Request — реализует поток данных запросов с применением пространства имен Dataset Object;
- Get All Dataset Request — заносит в запрос данные в массиве;
- Get All Workshop Request — заносит в запрос рабочие (измененные) данные в массиве;
- Login Request — запрос на вход в систему.

Пакет Response выполняет следующие функции:

- Dataset Response — реализует поток данных ответов с применением пространства имен Dataset Object;
- Get All Dataset Response — заносит в ответ данные в массиве;
- Get All Workshop Response — заносит в ответ все рабочие (измененные) данные в массив;
- Login Response — ответ на вход в систему.

Пакет Communication реализует следующие функции:

- Message Writer — инструмент для написания сообщений;
- Message Reader — инструмент для чтения созданных сообщений;
- Message Factory — класс, что преобразует, структурирует данные на основе запросов и ответов, формируя их в один пакет сообщений для удобной работы системы (преобразует данные в байт вид).

На этом пакет Tcp.core готов.

Пакет Client выполняет следующую функцию:

- реализует всю клиентскую базу, содержит метод входа в систему, получения и отправки данных под видом сообщений.

Пакет Gui выполняет следующую функцию:

- содержит FXML-файлы для описания интерфейса программы.

Пакет Controllers выполняет следующую функцию:

- осуществляет правильный процесс объединения элементов интерфейса и программных методов для обеспечения полного функционала программы.

Серверная часть

Серверная часть также содержит пакет Tcp.core.

Tcp.server — пакет — основа серверной части, выполняет следующие функции:

- ClientSession — реализует сессионный механизм пользователя и организует работу сервера, т. е. обмен сообщениями, хранение информации;
- Context — реализует механизм соединения;
- Server — сервер, содержит серверную информацию, отображает подключившихся;
- ServerLauncher — механизм запуска сервера;
- SessionManager — создает сессию.

Для сервера формируются программы `slp-client`, `slp-server`, `slp.tpc.server`, `SQLite`. Дополнительно необходимы программы конфигурации запуска, сборки итогового файла и приложения, соединений связи БД с `java`-программами.

После установки и запуска SQLite нажать «Новая база данных», выбрать корневой каталог (например, slp-server), ввести название db (рис. 4.4). В типе файла выбрать «Все файлы» и самостоятельно дописать файлу разрешение «s3db».

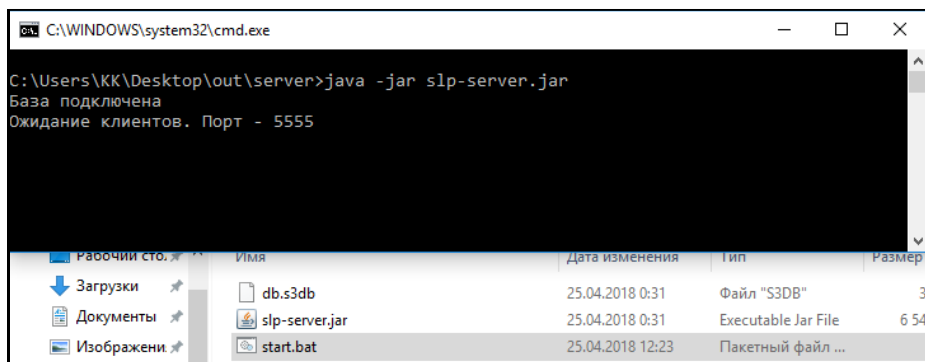


Рис. 4.4

Программный состав системы

Нажать «Сохранить» и закрыть появившееся окно создания таблицы. Перейти на вкладку SQL и вставить код.

```
BEGIN TRANSACTION;
DROP TABLE IF EXISTS `workshops`;
CREATE TABLE IF NOT EXISTS `workshops` (
  `id` INTEGER PRIMARY KEY AUTOINCREMENT,
  `name` INTEGER
);
DROP TABLE IF EXISTS `users`;
CREATE TABLE IF NOT EXISTS `users` (
  `id` INTEGER PRIMARY KEY AUTOINCREMENT,
  `name` TEXT NOT NULL UNIQUE,
  `password` TEXT NOT NULL,
  `role` INTEGER NOT NULL DEFAULT 0,
  `workshop` INTEGER,
  FOREIGN KEY(`workshop`) REFERENCES `workshops`(`id`)
);
INSERT INTO `users` VALUES (1,'admin','123',2,NULL);
DROP TABLE IF EXISTS `datasets`;
CREATE TABLE IF NOT EXISTS `datasets` (
  `id` INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
  `name` TEXT,
  `M` INTEGER,
  `N` INTEGER,
  `data` TEXT,
  `workshop` INTEGER,
  FOREIGN KEY(`workshop`) REFERENCES `workshops`(`id`)
);
INSERT INTO `datasets` VALUES (1,'test1',3,2,'1;2;3;4;5;6',NULL);
INSERT INTO `datasets` VALUES (2,'test2',NULL,4,NULL,NULL);
COMMIT;
```

Нажать Старт (F5), перейти на вкладку «Структура БД». Видно, что наша база заполнена.

В сервере необходимо открыть порт 5555 для создания сетевой связи. Ввести IP-адрес сервера

В командной строке WindowsCMDIP-Config ввести IP-адрес.

После запуска клиента ввести его IP-адрес.

Работа с системой

Установить пакет server на сервере, а пакет client — на клиенте.

Для запуска сервера необходимо запустить файл «start.bat» из папки «server» (рис. 4.5).

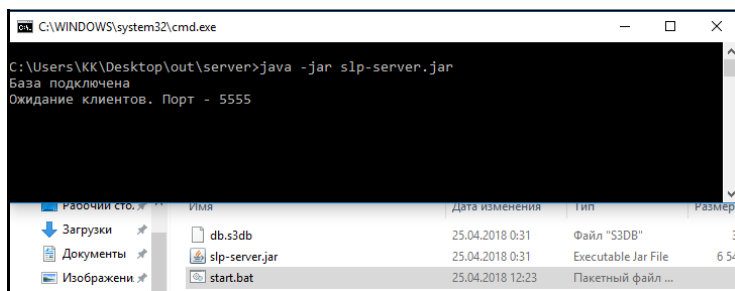


Рис. 4.5

Окно запуска сервера

Для запуска приложения следует перейти в папку «client» и запустить файл «slp_client.jar». Появится окошко авторизации пользователя (рис. 4.6).

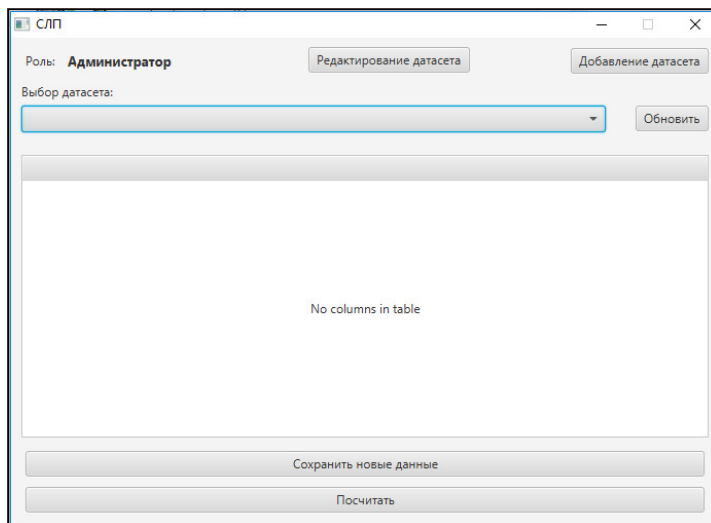


Рис. 4.6

Окно авторизации

В поле «Адрес сервера» ввести адрес машины-сервера: машины клиента и сервера находятся в одной сети.

Если вход осуществляется с машины, на которой расположен сервер, то поле «Адрес сервера» оставить пустым. В поле «Логин» и «Пароль» вводятся учетные записи пользователя, после чего открывается основной интерфейс программы (рис 4.7).

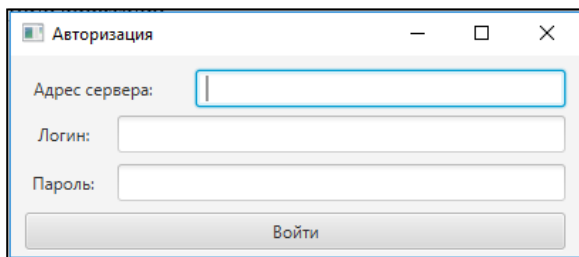


Рис. 4.7

Основной интерфейс программы

Учетные записи «Логин — пароль» имеют вид: администратор admin — 123; диспетчер — operator — 321; начальники цехов — foreman1 — 456, foreman2 — 678, foreman3 — 890.

Только администратор может добавлять новые таблицы в базу данных. Он может создать пустую таблицу или сразу заполнить её данными с помощью генератора (рис. 4.8).

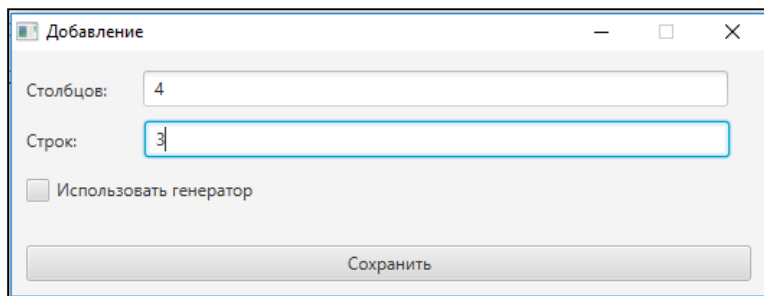


Рис. 4.8

Создание таблицы

Администратор может изменять существующий набор данных (таблицу) по нажатию кнопки «Редактирование датасета» (рис. 4.9).

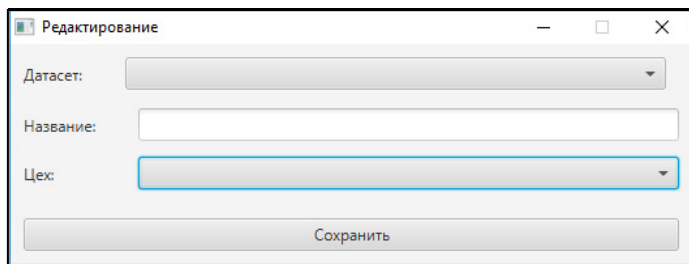


Рис. 4.9

Изменение таблицы

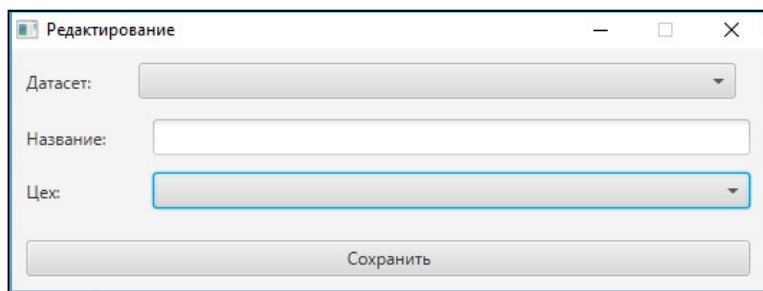


Рис. 4.10

Закрепление задач за начальником цеха

С помощью данного функционала он может назначить уникальное имя таблице и закрепить её за одним из начальников цехов (рис. 4.10).

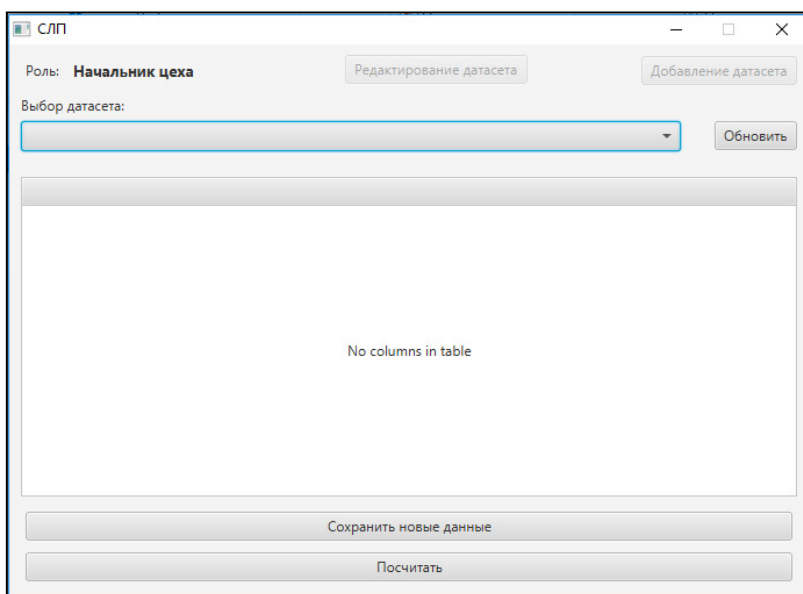


Рис. 4.11

Введение таблицы «Инструкция диспетчеру»

Администратор может самостоятельно осуществить введение таблицы или просто сохранить её в системе для дальнейшего решения (рис. 4.11).

Диспетчер (оператор) является связующим (промежуточным) звеном между исполнителями (начальниками цехов) и руководителем (администратором). Его полномочия сводятся только к закреплению конкретных таблиц за начальниками цехов. Осуществить это можно нажатием на кнопку «Редактирование датасета».

Инструкция начальнику цеха

Начальник цеха может работать только с назначенными ему таблицами. В его полномочия входят: расчёт закрепленных за ним таблиц, внесение в них

изменений (только данные), сохранение измененных таблиц под новым именем и дальнейшее их использование (рис. 4.12).

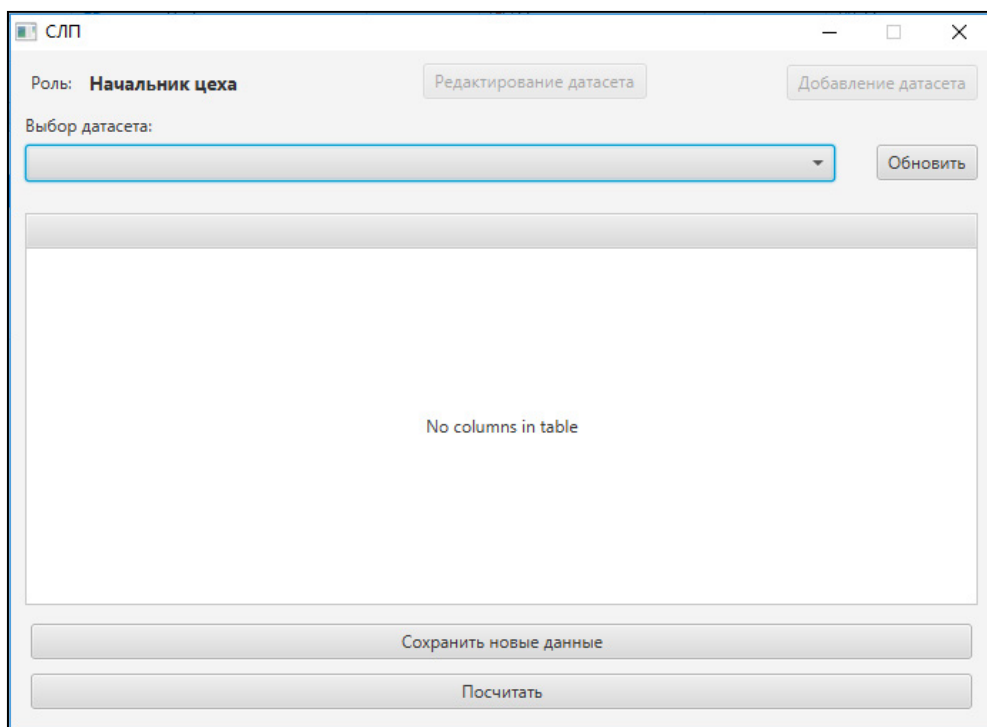


Рис. 4.12

Интерфейс начальника цеха

Содержание отчета

1. Титульный лист.
2. Задание.
3. Введение.
4. Описание задачи.
5. Экранная форма результата.
6. Заключение.